

PROTOTYPE PATTERN

Creational Design Pattern

Prototype Pattern

Imagine you're building a **chess game** where you need to save the current board position at different points (like an **undo feature** or **save points**). Instead of manually creating a fresh board and copying over every piece with its position one by one (which would be slow and complicated when you have 32 pieces to track), you can use the Prototype Pattern to simply clone the entire board.

The Prototype Pattern lets you make an exact copy of the current board, including all the pieces and their positions, without having to rebuild everything from scratch piece by piece.

Solution – Prototype Pattern

The **Prototype Pattern** is perfect for board games when you need to save the current game state (including where all the pieces are positioned) for features like undo/redo, save points, or creating a copy of the board for a new player.

Each chess piece has its own **clone method**, which means the entire board can be easily copied along with all the pieces in their current positions just one simple command instead of manually copying everything.

Benefits

1. **Easier Object Copying:** Instead of writing loops and manually copying every attribute, the `clone()` method handles everything in one simple call.
2. **No Need for Inheritance:** The pattern uses the `clone()` method to create copies, so the class itself manages how objects are duplicated – you don't need to create subclasses just for copying purposes.
3. **Flexible Copying Options:** You can choose between **shallow copy** (copying references) or **deep copy** (copying actual objects) depending on what your application needs.
4. **Fast Object Creation:** When you need to create objects with complex setups or when speed matters, the Prototype pattern lets you quickly duplicate existing objects instead of building new ones from scratch.